04 · Components

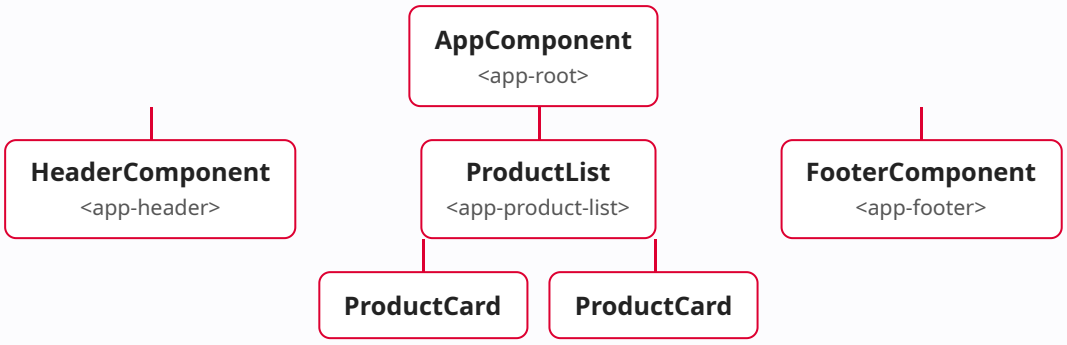
Beginner Angular learning notes

a. Definition

A **component** controls one part of the screen. It has three parts: a **class** (data and behaviour), a **template** (HTML) and **styles** (CSS). The `@Component` decorator ties them together and gives the component a **selector**, the custom HTML tag you write to use it.

To use one component inside another, add it to the parent's `imports` array and write its selector in the parent's template. Styles written in a component are **scoped**: they only affect that component.

b. Diagram



Components form a tree. A parent lists the children it uses in `imports`.

c. Syntax

```
@Component({
  selector: 'app-child',
  imports: [OtherComponent],     // components/pipes/directives used in the template
  templateUrl: './child.html',   // or inline: template: `...`
  styleUrl: './child.css',       // or inline: styles: `...`
})
export class ChildComponent {}
```

d. Example Program

`Programs/04_Components/profile-card.component.ts`

```
import { Component } from '@angular/core';

// A small reusable child component
@Component({
  selector: 'app-profile-card',
  template: `
    <div class="card">
      <h3>{{ name }}</h3>
      <p>{{ role }} - {{ city }}</p>
    </div>
  `,
  // Styles are scoped: they only apply inside this component
```

```
  styles: `
    .card { border: 1px solid #999; padding: 8px; margin: 6px; border-radius: 6px; }
    h3 { margin: 0; }
  `,
})
export class ProfileCardComponent {
  name = 'Priya';
  role = 'Frontend Developer';
  city = 'Coimbatore';
}

// The parent (root) component uses the child by its selector
@Component({
  selector: 'app-root',
  imports: [ProfileCardComponent], // <- must list every component used in the template
  template: `
    <h1>Our Team</h1>
    <app-profile-card />
    <app-profile-card />
  `,
})
export class AppComponent {}
```

e. Output

Rendered in the browser (text on screen):

```
Our Team
Priya
Frontend Developer - Coimbatore
Priya
Frontend Developer - Coimbatore
```

f. Real-World Example (English + தமிழ்)

English

A wedding hall has separate teams for catering, decoration and reception; each handles only its own job. Each Angular component handles one part of the screen.

தமிழ்

ஒரு திருமண மண்டபத்தை எடுத்துக்கொள்ளுங்கள். சமையல் பகுதி, அலங்காரம், வரவேற்பு, உணவு பரிமாறும் பகுதி என்று ஒவ்வொன்றுக்கும் தனிப் பொறுப்பாளர் இருப்பார். ஒவ்வொருவரும் தன் வேலையை மட்டும் பார்த்துக்கொள்வார்கள்; எல்லாம் சேர்ந்துதான் திருமணம் நடக்கும்.

Angular-இல் ஒவ்வொரு **component**-உம் அப்படியே ஒரு பொறுப்பாளர். Header, Footer, Product list என்று தனித்தனியாக எழுதி, `imports`-இல் சேர்த்து, பெரிய component-இல் பயன்படுத்துகிறோம். ஒரு பகுதியில் மாற்றம் செய்தால் மற்றவை பாதிக்கப்படாது - styles scoped ஆக இருப்பதால்.

g. Pros

- Reusable building blocks
- Easy to test in isolation
- Scoped styles avoid CSS clashes

h. Cons

- Too many tiny components can add overhead
- Need to import each component where it is used

i. Advanced / New Features

- Standalone by default (no NgModule)
- Self-closing tags like `<app-card />`
- `host` metadata replaces `@HostBinding` / `@HostListener`

j. Common Mistakes

- Forgetting to add the child component to the parent's `imports`; Angular then reports an unknown element.
- Using a selector that does not match the tag in the template (typos like `app-profilecard`).
- Putting too much code in one big component instead of splitting it.
- Expecting parent CSS to style the child's inside automatically (styles are scoped).

k. Interview Questions

Q1. What is a component?
A class decorated with `@Component` that controls a view through a template and styles.

Q2. What is a selector?
The custom HTML tag (or attribute) used to place a component in a template.

Q3. What does `standalone` mean?
A standalone component declares its own `imports` and needs no NgModule. It is the default since Angular 19.

Q4. What is view encapsulation?
By default Angular scopes component CSS so it does not leak out or in (emulated shadow DOM).

l. Practice Exercises

1. Make a `product-card` component showing a name and price, and display it three times.
2. Add a header and footer component and compose them in the root component.
3. Move the template and CSS of `profile-card` into separate files using `templateUrl` and `styleUrl`.